
deep reinforcement learning for hpc job scheduling

A statistical evaluation of six DRL treatments, three production heuristics and a masked random control, across two real Slurm workload traces.

PRESENTER

Justin M. Cheney · 4323819

SUPERVISOR

Prof. Clement Nyirenda

INSTITUTION

University of the Western Cape

what is hpc ■

A supercomputer is one machine shared by hundreds of researchers at once.

You don't get your own. You submit work into a queue and wait your turn alongside everyone else's.

What changed is the **hardware underneath**. Clusters used to be uniform — every node the same. Now they carry GPUs and accelerators, and different jobs need very different resources from very different nodes.

Half of the world's fastest 500 machines now use accelerators. Twenty years ago it was one machine in five hundred.

SHARE OF THE TOP500 USING ACCELERATORS · 2006–2025



Source: TOP500 semi-annual lists, aggregated. Nov 2025: 253 of 500 systems.

what is job scheduling ■

Something has to decide who goes next. That something is the scheduler.

You have waited for it. When your job sat in the queue for six hours, a scheduler put it there.

IT DOES TWO JOBS

Queue ordering — which waiting job runs next.

Resource allocation — which nodes it lands on.

This study holds allocation fixed and changes only the ordering, so any difference measured is the scheduling policy and nothing else.

HOW PRODUCTION SYSTEMS DECIDE TODAY

FCFS	First come, first served. Fair, but one big job blocks everyone behind it.
SJF	Shortest job first. Lowers average waiting, starves long jobs.
LCFS	Last come, first served. Favours whoever arrived most recently.
UNICEP	A priority score combining job size and age.

These are **fixed rules**. Somebody wrote them down once, and they do the same thing forever. They are cheap, predictable, and that is exactly why every production cluster still runs them.

They also cannot adapt when the hardware and the workload stop being uniform.

what is deep rl

Instead of writing the rule down, you let a neural network learn one by trial and error.

The agent looks at the queue, picks a job, and gets a score telling it whether that was a good idea. Repeat a few million times and the network settles into a policy.

THE SCORE I GIVE IT — THE ENTIRE REWARD FUNCTION

```
# negative mean queued waiting time, in hours
reward = -avg_waiting / 3600.0
```

Shorter queues score higher. That single line is the only thing any of these agents is trying to do — remember it, because it comes back at the end.

— Point that at a queue and the fit is unusually neat

THREE FAMILIES, THREE WAYS OF LEARNING THE SAME THING

FAMILY	ALGORITHM	LEARNS BY
Value-based	DQN	scoring every option
Policy gradient	PPO	adjusting the policy
Actor–critic	A2C	both at once

They make genuinely different assumptions about exploration, stability and how much experience they need.

A comparison confined to one family cannot speak for the others — which is the problem on the next slide.

how they combine ■

LITERATURE

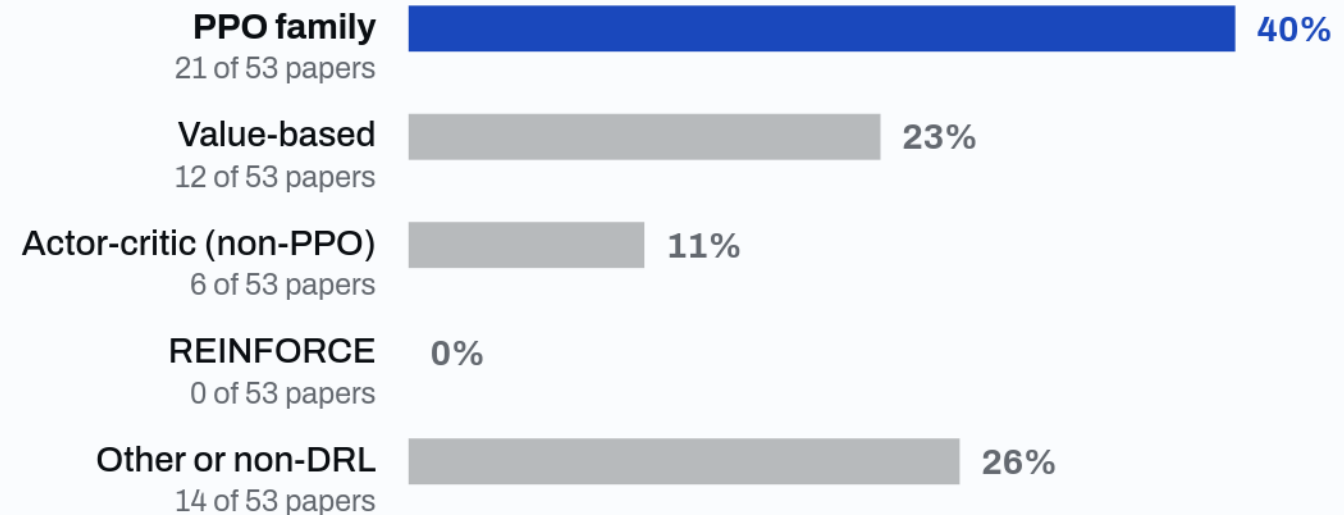
The queue becomes the **state**. Picking a job becomes the **action**. Waiting time becomes the **reward**. Scheduling turns into a reinforcement learning problem cleanly, which is why the field moved this way.

AND ONE DETAIL EVERYBODY SHIPS

Action masking. At each step, jobs that cannot possibly run — nothing free to run them on — are hidden from the agent before it chooses.

It is near-universal, and it is reported as an implementation detail rather than tested as a variable. So published numbers cannot separate what the learning contributed from what the mask contributed.

WHICH ALGORITHM FAMILY 53 SURVEYED PAPERS USED



THE GAP

no primary study evaluates all three families together

what i'm asking ■

RQ1

What does action masking actually contribute?

How much of the published performance comes from the learned policy, and how much from the mask that makes it runnable at all?

RQ2

Is the field's concentration on PPO justified?

And if it is, on which axis — schedule quality, or cost?

RQ3

When is a DRL scheduler worth its inference cost?

Against a heuristic that is effectively free to run, under what workload conditions does training one pay for itself?

To answer the first one I need something the literature does not usually include: a control that does no learning at all.

A masked random policy — it picks uniformly at random from exactly the jobs the mask leaves available. Same mask, same seeds, no learning. It measures what the mask alone is worth.

TEN TREATMENTS, TWO REAL TRACES, TEN SEEDS

SIX DRL TREATMENTS

MaskablePPO · MaskableDQN · MaskableA2C	masked
PPO · DQN · A2C	unmasked
FOUR REFERENCE CONDITIONS	
SJF · LCFS · UNICEP	heuristics
Random, masked	control

Each algorithm appears with *and* without the mask, so masking is an experimental factor rather than an assumption. 120 runs in total.

Statistics follow the framework of Carrasco et al. — rank-based throughout, because performance across seeds is not normally distributed.

MEASURING INFERENCE COST — TIMED AROUND THE DECISION ONLY

```
t0 = time.perf_counter()
action, _ = model.predict(obs, deterministic=True,
                          action_masks=action_masks)

latencies.append(time.perf_counter() - t0) # decision only
obs, reward, done, truncated, _ = env.step(action)

decision_latency_mean_ms = np.mean(latencies) * 1000.0
```

The clock brackets `model.predict` alone — the simulator step is outside it — so this is the cost of the policy thinking, not the cost of simulating the cluster.

WHAT ONE RUN PRODUCES

```
run_id, algorithm, use_masking, seed, split_id,
episode_reward, decision_count, decision_latency_mean_ms,
eval_wall_s, avg_waiting, avg_slowdown, cpu_utilization, ...

{ "git_commit": "fae1c73...",
  "manifest_sha256": "e9cac27a..." }
```

One row per run, 60 rows per split, each stamped with the commit and a hash of its inputs.

what the results say

FIRST: HOW MUCH ROOM WAS THERE TO IMPROVE?

3.7%

Physical trace. Gap between the best heuristic and the random control. Almost nothing to win.

38.3%

Deeplearn trace. Ten times more room for a learned policy to show a benefit.

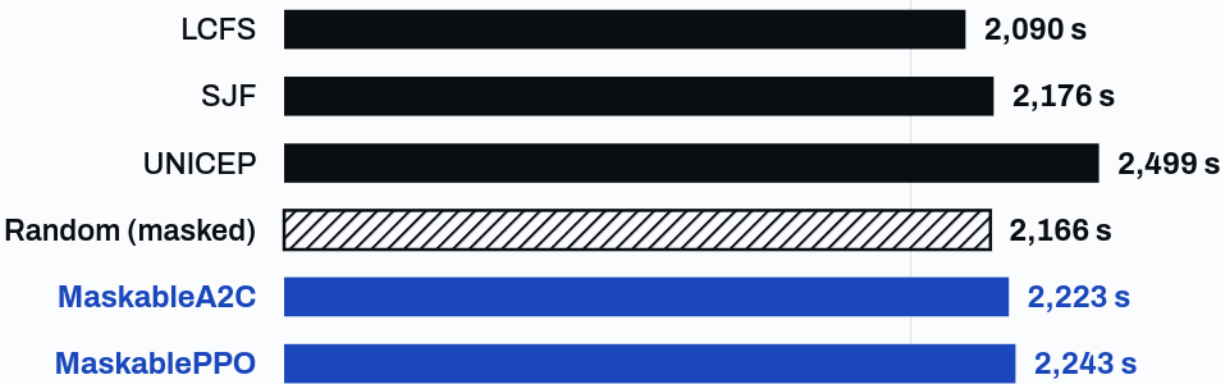
0 of 6

Physical. No DRL treatment beats the random control. Seven of nine comparisons favour random.

10 of 10

Deeplearn. MaskablePPO beats the control on every primary metric, on every seed.

PHYSICAL · AVERAGE WAITING TIME



■ masked DRL ■ heuristic ▨ random control

DEEPLearn · AVERAGE WAITING TIME



MaskablePPO also cuts **maximum** slowdown by 22.0% against LCFS and 63.2% against the control — it handles the awkward jobs better, not the average one.

AND BETWEEN THE ALGORITHMS THEMSELVES **nothing separates from anything else**

Two traces that disagree, until you look at the headroom

what that means ■

RQ1 · MASKING BUYS DEPLOYABILITY, NOT SCHEDULE QUALITY

Without the mask, an agent can pick a job that cannot run. The simulator advances the clock, the queue drains, and time freezes on an identical state. **Not one unmasked run ever finished an episode** — all ten seeds, both traces, stopped by a hang guard at 500,000 steps.

Masking removes that failure mode. It does not prevent divergence — MaskableDQN still blew up.

RQ2 · THE CONCENTRATION ON PPO IS NOT A QUALITY RESULT

A PPO treatment topped both traces and separated from the leaders on neither. What PPO does have is cost: it trained **9–22% faster** than the A2C variants.

Is a 9–22% margin on training time worth a 40% concentration of the field?

RQ3 · WORTH ITS COST ONLY IN THE TAIL, AND ONLY SOMETIMES

MaskablePPO ran **13–35× slower** than the fastest heuristic. It earns that only on heterogeneous workloads, and only where tail fairness is the objective.

— What the mask did, what PPO was worth, and what bounds both

512 actions between reward signals	0.0058 what the next reward is worth at $\gamma = 0.99$	82× gap in gradient updates, PPO vs A2C
---	---	---

Two things bound every number in this talk. That one-line reward fires once every 512 decisions, so the signal is far sparser than the horizon it is discounted over. And the same three million steps buy PPO about 7,300 updates and A2C about 600,000, under inherited defaults.

The diagnostics say the agents *optimised successfully*. They were optimising the wrong thing.

So the parity I report is a lower bound, not a verdict on DRL.

THE CLAIM I'D DEFEND

a masked random control is cheap, decisive, and belongs in this literature

Without it, the parity my masked agents reach with production heuristics on the homogeneous trace would have been reported as a success. It is the difference between “the policy learned something” and “the mask was doing the work”.

Masking supplies deployability rather than schedule quality; no algorithm family separates on quality; and the cost is justified only in the tail of heterogeneous workloads.

WHERE IT GOES NEXT

Fix the reward	Denser than one signal per 512 actions.
Equalise the budget	Match gradient updates, not just environment steps.
A trace gradient	Vary heterogeneity at fixed reward, to separate headroom from misalignment.
Distillation	Bring inference cost down toward heuristic latency.

Thank you.

Justin M. Cheney · 4323819
Supervised by Prof. Clement Nyirenda
Computations on UWC's HPC and Research Cloud facilities.

references

SOURCES CITED IN THIS TALK

SYSTEMS & WORKLOADS

- Yoo, Jette & Grondona (2003).** SLURM: Simple Linux Utility for Resource Management. JSSPP, LNCS 2862, 44–60.
- Lifka (1995).** The ANL/IBM SP Scheduling System. JSSPP, LNCS 949, 295–303. Backfilling.
- Henry (1984).** The UNIX System: The Fair Share Scheduler. AT&T Bell Labs Tech. J. 63(8), 1845–1857.
- TOP500 (2026).** TOP500 List Statistics. top500.org/statistics/list — accelerator share, slide 2.

REINFORCEMENT LEARNING

- Sutton & Barto (2018).** Reinforcement Learning: An Introduction, 2nd ed. MIT Press.
- Mnih et al. (2015).** Human-level control through deep reinforcement learning. Nature 518, 529–533. DQN.
- Mnih et al. (2016).** Asynchronous Methods for Deep Reinforcement Learning. ICML, PMLR 48, 1928–1937. A2C/A3C.
- Schulman et al. (2017).** Proximal Policy Optimization Algorithms. arXiv:1707.06347. PPO.

DRL FOR SCHEDULING, AND MASKING

- Zhang, Dai, He, Bao & Xie (2020).** RLScheduler: An Automated HPC Batch Job Scheduler Using RL. SC20.
- Wang, Rodriguez & Lipovetzky (2025).** Optimizing HPC scheduling: a hierarchical RL approach. J. Supercomputing 81(8).
- Stolz et al. (2024).** Excluding the Irrelevant: Focusing RL through Continuous Action Masking. NeurIPS 37, 95067–95094.
- Raffin et al. (2021).** Stable-Baselines3. JMLR 22(268), 1–8. With sb3-contrib for the maskable variants.

STATISTICS & REPRODUCIBILITY

- Carrasco, García, Rueda, Das & Herrera (2020).** Recent trends in the use of statistical tests for comparing algorithms. Swarm Evol. Comput. 54, 100665. The framework followed here.
- Demšar (2006).** Statistical Comparisons of Classifiers over Multiple Data Sets. JMLR 7, 1–30. Friedman and the CD diagram.
- Holm (1979).** A simple sequentially rejective multiple test procedure. Scand. J. Stat. 6(2), 65–70.
- Mölder et al. (2025); Dolstra et al. (2004).** Snakemake; Nix. F1000Research 10(33); USENIX LISA. The run pipeline.